VOLCANO-CA-MC: DOKUMENTACJA TECHNICZNA

Interfejs graficzny

Bartosz Pajda

Wygenerowano: 22.10.2026

Wprowadzanie danych, uruchamianie obliczeń, wizualizacja 2D i 3D.

Stan na 09.10.2026. Projekt ma interfejs użytkownika w przeglądarce (aplikacja web, lokalna). Parametry symulacji ustawia się w formularzu. GUI zapisuje z nich plik konfiguracyjny, tworzy folder na wyniki i uruchamia silnik C++ jako osobny proces. Postęp widać na liście zadań, a wynik jako przekroje 2D i rzut 3D ścian zewnętrznych bloku (oraz powierzchnię wulkanu i woksele lawy). Silnik dostał trzy małe opcje dla GUI (`--print-defaults`, `--validate`, `--progress`). Algorytmy CA i MC się nie zmieniły: wyniki są bitowo identyczne z wynikami sprzed GUI (punkt 6).

Stack: backend **Python 3.12 + FastAPI + uvicorn + numpy**, nasłuch tylko na `127.0.0.1`; frontend **HTML + JavaScript (moduły ES) bez kroku budowania**, **three.js r160** z `OrbitControls` (3D), canvas 2D (przekroje), **Chart.js 4.4.1** (wykresy). Biblioteki są skopiowane do `gui/static/vendor/` (wersje i licencje: `gui/static/vendor/README.md`), więc GUI działa bez internetu. Interfejs jest po polsku.

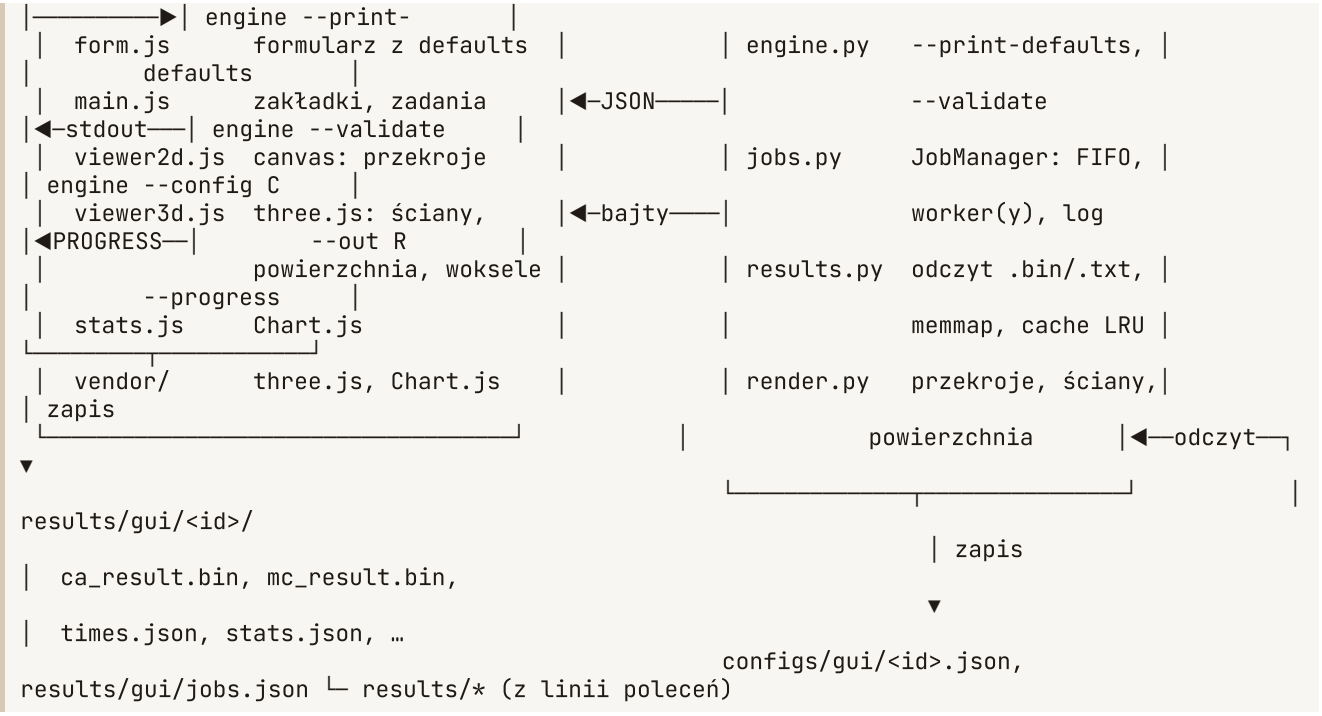
Uruchomienie jedną komendą (z katalogu projektu): `python -m gui` albo dwuklik `gui\run_gui.bat`: szczegóły w punkcie 4.

1. Funkcje → realizacja

Funkcja	Realizacja	Ekran GUI	Kod
Pobranie parametrów z GUI	Formularz ze wszystkimi parametrami silnika, pogrupowany jak config (Przestrzeń, Ogólne, Teren, Kratery/zarodki, Automat CA, Monte Carlo, Wyjście). Formularz powstaje z `engine --print-defaults`, więc ma dokładnie te klucze i wartości domyślne co silnik. Pola mają jednostki i podpowiedzi, wartości wyliczeniowe to listy wyboru. Pola, które w danym trybie nie mają znaczenia, są wyszarzone (np. teren w trybie `classic`, mobilność w MC `full`). Presety to pliki z `configs/*.json` (oraz configi poprzednich zadań). Do tego import configu z pliku, eksport formularza do JSON, przycisk **Sprawdź** (walidacja silnikiem, błędy po polsku przy polach) i szacunek pamięci i czasu	„Nowa symulacja”	`gui/static/js/form.js`, `main.js`, `estimate.js`; `gui/engine.py`; `engine/src/app/ConfigCheck.*`
Utworzenie plików konfiguracyjnych	Po **Uruchom** backend zapisuje config do `configs/gui/<id>.json`, gdzie `<id>` = data-czas + bezpieczna nazwa zadania, np. `20261009-195824_demo_test_3d_5_kraterow`. Ścieżka pojawia się pod przyciskiem i na liście zadań. Metadane GUI trafiają do klucza `_gui`; silnik pomija klucze zaczynające się od `_`	„Nowa symulacja” (komunikat po uruchomieniu), „Zadania” (kolumna Pliki)	`gui/jobs.py` (`JobManager.create`)
Utworzenie folderów na wyniki	Folder `results/gui/<id>/` tworzy GUI przed startem silnika. Trafiają do niego wyniki silnika, a od GUI dodatkowo `engine.log` (wyjście programu) i `gui_job.json` (nazwa zadania)	jw.	`gui/jobs.py`
Wizualizacja końcowej mikrostruktury: 2D	Przekrój ⊥ osi x / y / z z suwakiem warstwy (dla 2D jedyna płaszczyzna siatki). Kolorowanie: typ komórki, krater/ziarno (`flowId`), temperatura, kryształ (MC), czas stygnięcia. Legenda, granice ziaren, zoom kółkiem i przesuwanie (piksele ostre), podpowiedź z wartościami komórki, eksport PNG	„Podgląd 2D”	`gui/static/js/viewer2d.js`, `palette.js`; `gui/results.py`, `render.py`
Wizualizacja: 3D (może być rzut samych ścian zewnętrznych)	**Prostopadłościan nx × ny × nz z 6 ścianami zewnętrznymi** oteksturowanymi wybranym polem (proporcje jak `nx:ny:nz`, osie z podpisami, ta sama paleta i legenda co w 2D). Do tego **powierzchnia** (najwyższa niepusta komórka w każdej kolumnie, czyli zbocze wulkanu z jęzorami lawy) i **woksele** lawy/bazaltu (`InstancedMesh`, limit 150 tys.). Obrót, przesuwanie, powiększenie, reset kamery, eksport PNG	„Podgląd 3D”	`gui/static/js/viewer3d.js`
Czasy i statystyki	zakładka „Czasy i statystyki”: tabela `times.json`, pamięć, pliki, `stats.json` (CA i MC), wykresy liczby ziaren i średniej wielkości ziarna vs MCS (Chart.js), historia płynnej lawy w krokach CA	„Czasy i statystyki”	`gui/static/js/stats.js`

2. Architektura

```
 przeglądarka (http://127.0.0.1:8000)                backend Python (gui/)
silnik C++
 ┌──────────────────────────────────┐   HTTP   ┌──────────────────────────────────┐ podproces
┌──────────────────────┐
 │ index.html + js/*.js (moduły ES)  │──JSON──▶│ app.py      FastAPI, /api/...
```



Backend niczego nie liczy: zarządza plikami i procesami i wycina z wyników to, co trzeba pokazać. Całe obliczenia robi `engine.exe`. Do przeglądarki nigdy nie idzie cały blok (300^3 = 27 mln komórek), tylko przekrój, 6 ścian, mapa powierzchni albo ograniczona liczba wokseli.

Zadanie od formularza do wyniku

1. **Sprawdź:** `POST /api/validate` → config do pliku tymczasowego → `engine --validate --config tmp.json` → `{"ok", "errors", "warnings"}`. Błędy po polsku wracają do formularza, a pola z błędem są podświetlone.
2. **Uruchom:** `POST /api/jobs {name, config}`. Najpierw ta sama walidacja (błąd → HTTP 422 i nic nie powstaje), potem `configs/gui/<id>.json`, `results/gui/<id>/` z `gui_job.json`, wpis w `results/gui/jobs.json` i dopisanie do kolejki FIFO.
3. **Worker** (wątek; domyślnie 1, liczba w `gui/settings.json` → `workers`) bierze pierwsze zadanie z kolejki i uruchamia `engine --config configs/gui/<id>.json --out results/gui/<id> --progress` z katalogiem roboczym `volcano-ca-mc`. Linie `PROGRESS …` aktualizują postęp zadania. Pozostałe wyjście (stdout + stderr: podsumowanie, ostrzeżenia, błędy) trafia do `engine.log`.
4. Po zakończeniu procesu: status `gotowe` (kod 0), `błąd` (ostatnia linia „Błąd…” z logu) albo `anulowane`. Do tego czas trwania i kod wyjścia. Stan jest zapisywany w `jobs.json` (zapis atomowy).
5. Lista zadań w przeglądarce odświeża się co 1 s, gdy coś jest w kolejce (inaczej co 5 s). Po zakończeniu zadania odświeża się lista wyników, a „Pokaż wynik” otwiera podgląd 2D.

Statusy: oczekuje / działa / gotowe / błąd / anulowane / przerwane. Po restarcie GUI zadanie, które „działało”, dostaje status **przerwane**, a oczekujące wracają do kolejki. **Przygotowanie pod serie zadań:** kolejka FIFO z konfigurowalną liczbą workerów już jest (`JobManager.submit()`). Generator serii doda po prostu N zadań, a kilka zadań uruchomionych naraz wykonuje się po kolei (widać to na zrzucie z punktu 3). Backend obsługuje też `mcFrom` (samo MC na gotowym wyniku CA, `engine --mc-from`): w formularzu to pole „Źródło danych”.

Endpointy

Metoda i adres	Opis
`GET /api/info`	ścieżka silnika (i skąd wzięta), katalogi, liczba workerów, ostrzeżenia
`GET /api/defaults`	pełny domyślny config (`engine --print-defaults`)
`GET /api/presets`	`configs/*.json` + 30 ostatnich `configs/gui/*.json` (nazwa, opis `_opis`, config)

Metoda i adres	Opis
`POST /api/validate`	`{config, mcFrom?}` → `{ok, errors[], warnings[]}`
`POST /api/jobs`	`{name, config, mcFrom?}` → zadanie (201) albo `{detail, errors[], warnings[]}` (422)
`GET /api/jobs`, `GET /api/jobs/{id}`	zadania: status, postęp, czas, kod wyjścia, ścieżki configu i wyników
`GET /api/jobs/{id}/log?tail=N`	ostatnie linie `engine.log`
`POST /api/jobs/{id}/cancel`	anulowanie: usunięcie z kolejki albo zabicie procesu
`GET /api/results`	wyniki z `results/*` (linia poleceń) i `results/gui/*` (GUI): wymiary, tryb, sąsiedztwo, kroki, czasy, pliki
`GET /api/result?id=`	metadane: wymiary, płaszczyzna 2D, dostępne pola, zakresy wartości (stałe skale kolorów), źródła danych
`GET /api/result/details?id=`	`times.json`, `stats.json`, `mc_stats.csv` (kolumny), `config_used.json`
`GET /api/result/slice?id&field&axis&index`	przekrój (bajty)
`GET /api/result/faces?id&field&top=layer|surface`	6 ścian zewnętrznych jednym żądaniem (bajty)
`GET /api/result/surface?id&field`	mapa powierzchni: `height` = indeks z najwyższej niepustej komórki + 1, albo pole na tej wysokości (bajty)
`GET /api/result/voxels?id&field&limit`	woksele lawy/bazaltu: `uint16 xyz[k·3]` + wartości (bajty)
`GET /api/result/cell?id&x&y&z`	wszystkie pola jednej komórki (podpowiedź w 2D)

Interaktywna dokumentacja API (OpenAPI) jest pod `/api/docs`.

Format przesyłania danych. Tablice idą jako surowe bajty `application/octet-stream` (little-endian, wiersz po wierszu: wiersz 0 = dół płaszczyzny). Opis jest w nagłówku `X-Meta` (JSON): `dtype` (`uint8` typ, `int32` flowId/kryształ, `float32` temperatura, `uint16` czas stygnięcia i wysokość), `width`, `height`, krok próbkowania, nazwy osi; dla ścian dodatkowo przesunięcie każdej ściany w buforze. Przeglądarka tworzy z bajtów `TypedArray` i koloruje sama, więc zmiana palety nie wymaga nowego żądania. Przekrój 150×80 z `etna_3d` to 12 KB (typ) + 48 KB (`flowId`). Ten sam przekrój jako JSON byłby kilka razy większy i wolniejszy do sparsowania.

Pola. `type`, `flowId`, `temp` (z `ca_result.*`), `crystal` (z `mc_result.*`), `cooling` = czas w stanie płynnym `solidStep - arrivalStep` (tylko `ca_result.bin` v2; dla lawy, która nie zastygła: ostatni krok CA – `arrivalStep`, tak samo jak w MC i `plot_slice.py`). Wynik `--mc-from` bierze typy komórek z folderu wejściowego (`_mcFrom` w `config_used.json`). Wynik MC `full` (bez CA) traktuje domenę MC jako bazalt.

Duże wyniki. Pliki `.bin` większe niż 128 MB są mapowane w pamięć (`np.memmap`), więc przekrój czyta z dysku tylko potrzebne strony. Mniejsze są wczytywane do RAM: Windows nie pozwala nadpisać zmapowanego pliku, a foldery w `results/` bywają nadpisywane z linii poleceń w trakcie oglądania. Płaszczyzny większe niż **4 mln komórek** są **próbkowane** co `s` komórek w obu osiach, gdzie s = ⌈√(komórki / 4 mln)⌉ (np. 4000×4000 → co 2.). Krok jest w `X-Meta` i na pasku stanu, a współrzędne w podpowiedzi uwzględniają próbkowanie. Powierzchnia w 3D ma najwyżej 360 tys. wierzchołków (co s-ta kolumna), woksele 150 tys. (co k-ty). Wyniki tylko w `.txt` (starsze uruchomienia) są parsowane `np.loadtxt` (~0,6 s na 1 mln wierszy; `np.fromfile(sep=" ")` potrzebował 16 s), do 30 mln komórek. Wczytane wyniki trzyma pamięć podręczna LRU (4 ostatnie), unieważniana po zmianie rozmiaru lub daty plików. Zakresy wartości (temperatura, `flowId`, kryształy, czas stygnięcia) są liczone raz na wynik, warstwami, więc memmap nie trafia cały do RAM.

Kolory

> **Później GUI dostało nową paletę „Bazalt i lawa”** (skała ciepła szarość-brąz, lawa pomarańcz-żar, bazalt grafit, kratery z palety sprawdzonej pod kątem daltonizmu, temperatura w skali „żar”, czas stygnięcia granat → turkus; motyw ciemny i jasny); opis i uzasadnienie w dokument „Wygląd interfejsu”. Zrzuty `img/gui_*.png` w tym dokumencie pokazują już nowy wygląd. Poniżej pierwotna paleta z pierwszej wersji GUI (została w obrazkach z `tools/plot_slice.py`).

Paleta była wspólna z `tools/plot_slice.py`: powietrze = tło `#fcfcfb`, skała `#c3c2b7`, lawa `#eb6834`, bazalt `#2a78d6`. Kratery 1–8 mają stałe sloty palety kategorycznej w tej samej kolejności co `plot_slice.py`. Więcej ziaren (rozrost `classic`) i kryształy MC dostają kolor wyliczany z ID hashem Knutha, dokładnie wzór `crystal_rgb()` z `plot_slice.py`. Ten sam ID ma więc ten sam kolor na każdym przekroju, ścianie 3D i obrazku z narzędzia. Temperatura to jedna barwa pomarańczowa (jasny = chłodniej), czas stygnięcia: jedna barwa niebieska (ciemny = długo płynna lawa → duże kryształy). Skala obu jest wspólna dla całego wyniku, a nie dla jednego przekroju. W rzucie z góry skała jest cieniowana wysokością (jak `--axis top` w `plot_slice.py`). Granice ziaren przy powiększeniu ≥ 5 px/komórkę są liniami między komórkami, przy mniejszym przyciemnionymi komórkami granicznymi.

Szacunek przed uruchomieniem (heurystyka z profilu aplikacji)

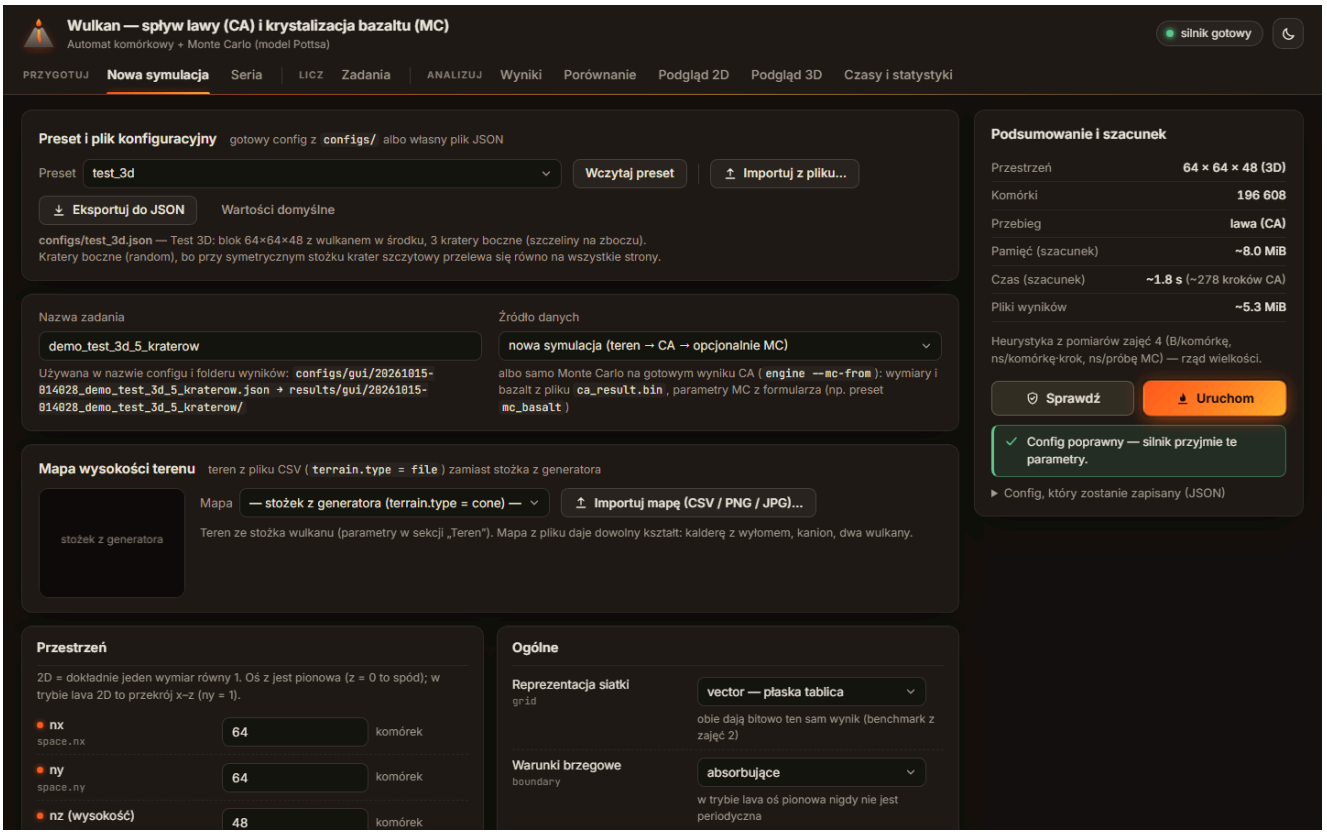
Panel „Podsumowanie i szacunek" liczy na bieżąco (`estimate.js`, dane z `docs/profilowanie.md`), gdzie N to liczba komórek, a Nf = (nx+2)(ny+2)(nz+2): - **pamięć** z modelu `MemoryModel` (zgodność z pomiarem 0–4%): CA `26·Nf + 14·N` B (3D ≈ 40 B/komórkę, 2D ≈ 91 B przez ramkę w nieużywanej osi), MC `full` `5·Nf + 37·N`, lawa + MC: CA + `5·Nf + 8·N`; - **czas** = koszt × komórki × kroki: CA classic 7 (VN 3D) / 9 (VN 2D) / 11 (Moore 2D) / 20 (Moore 3D) ns/(komórka·krok), lawa 32 ns. Liczba kroków classic ≈ promień największej luki między zarodkami (VN: (nx+ny+nz)/2, Moore: max(n)/2, podzielone przez zarodki^(1/d)); lawa: górny szacunek `eruptionSteps + 2·max(n)`. MC: 40–250 ns/próbę zależnie od sąsiedztwa i tego, czy siatka mieści się w pamięci podręcznej (domena `basalt` ≈ 5% N); - **pliki**: bin 13 B/komórkę (MC 4 B), txt ≈ 15–21 B (MC ≈ 13 B), plus migawki.

Ostrzeżenia: pamięć > 2 GB (żółte), > 8 GB (czerwone), czas > 10 min, txt dla > 10 mln komórek (zalecany bin), siatka 2D > 1 mln komórek (ramka zajmuje 2/3 pamięci). To szacunek rzędu wielkości; dla lawy czas zależy od parametrów stygnięcia (ile kroków potrwa spływ), a na innym komputerze może się różnić kilkukrotnie. Przykład: dla `test_3d` heurystyka daje ~1,8 s (278 kroków), a pomiar 1,7 s CA (284 kroki).

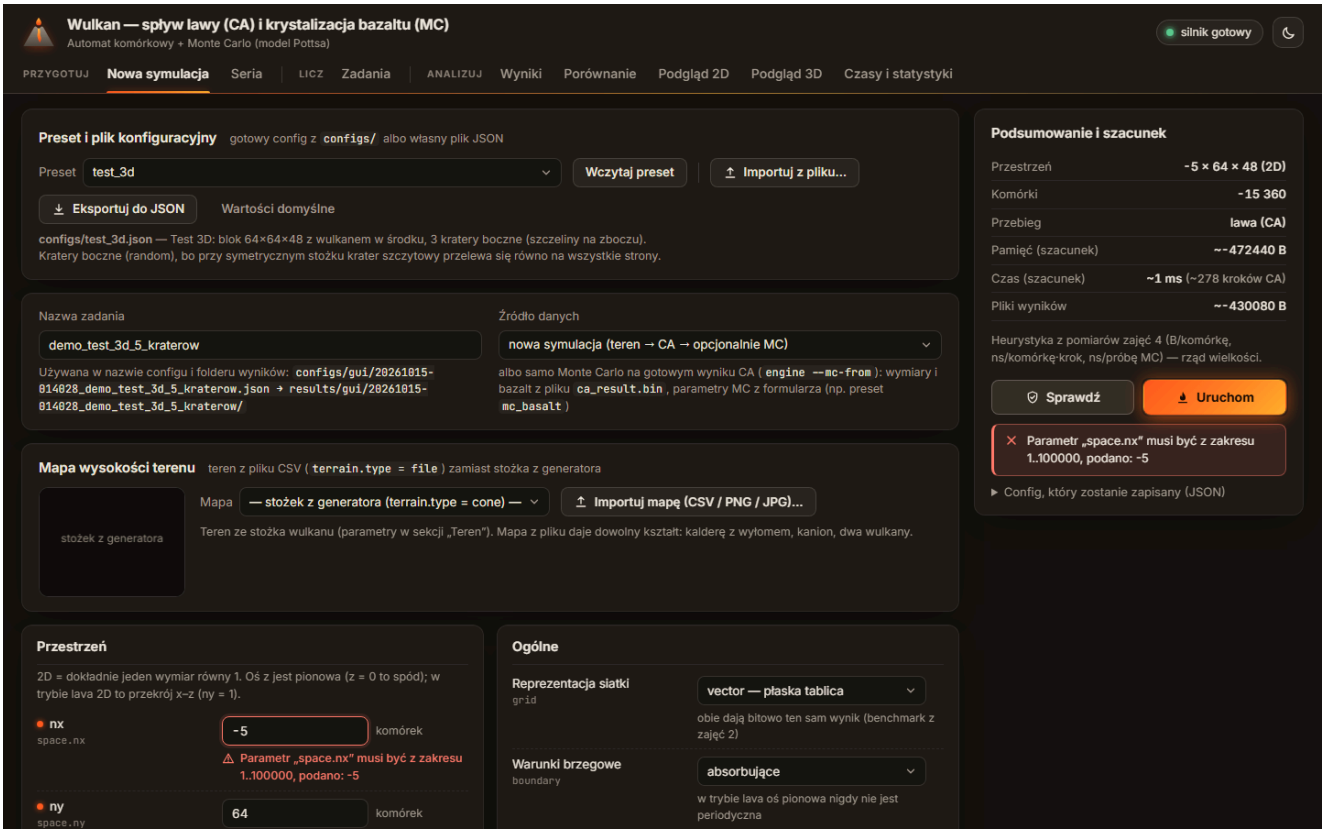
3. Zrzuty ekranu

Wszystkie zrzuty zrobił skrypt `gui/tests/screenshots.py` (Playwright + Chromium, okno 1440×900) w czasie scenariusza z punktu 6. Zadania `demo_*` powstały w nim z prawdziwych presetów.

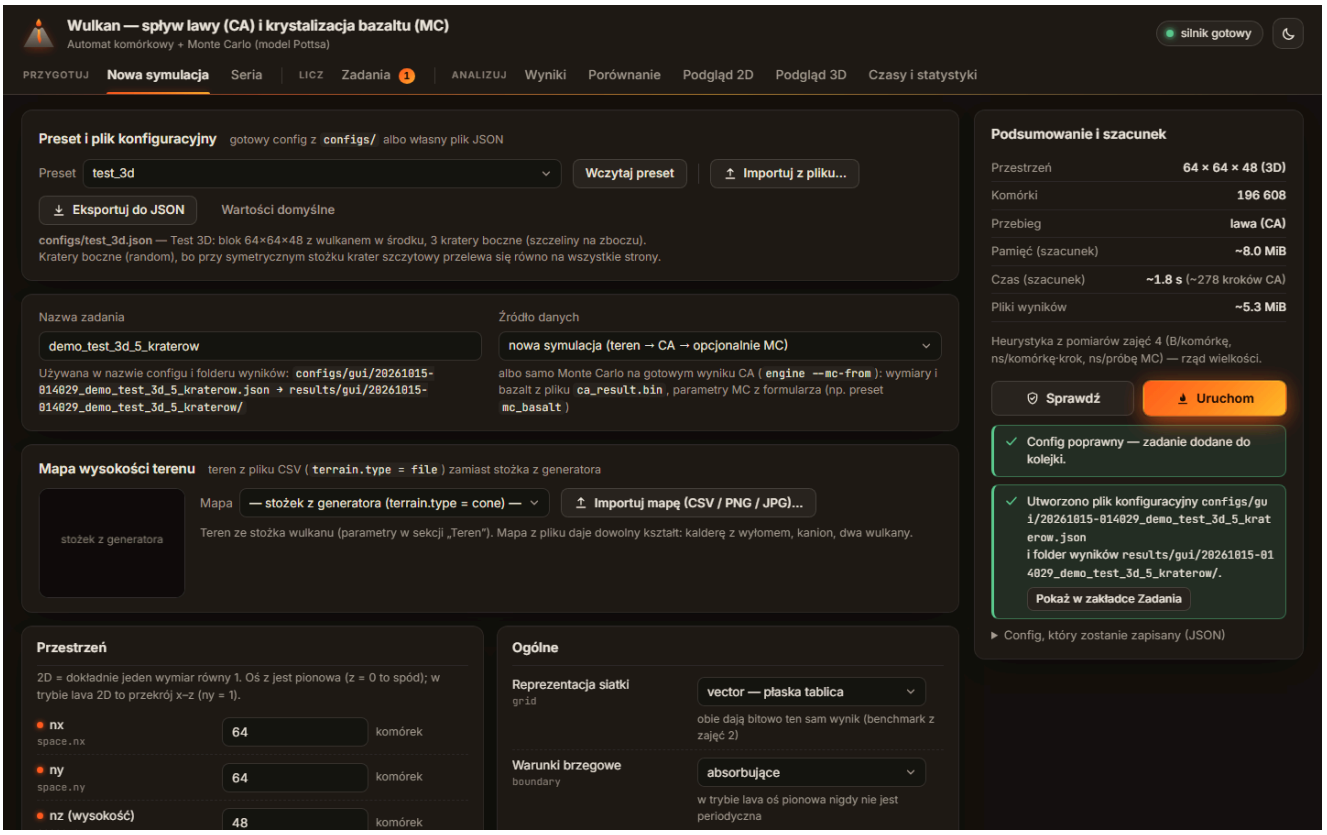
Nowa symulacja: preset `test_3d`, zmieniona nazwa i liczba kraterów (5), po „Sprawdź". Niebieska kropka przy etykiecie oznacza wartość inną niż domyślna. Pola terenu, kraterów i CA są aktywne, a parametry MC wyszarzone (MC wyłączone):



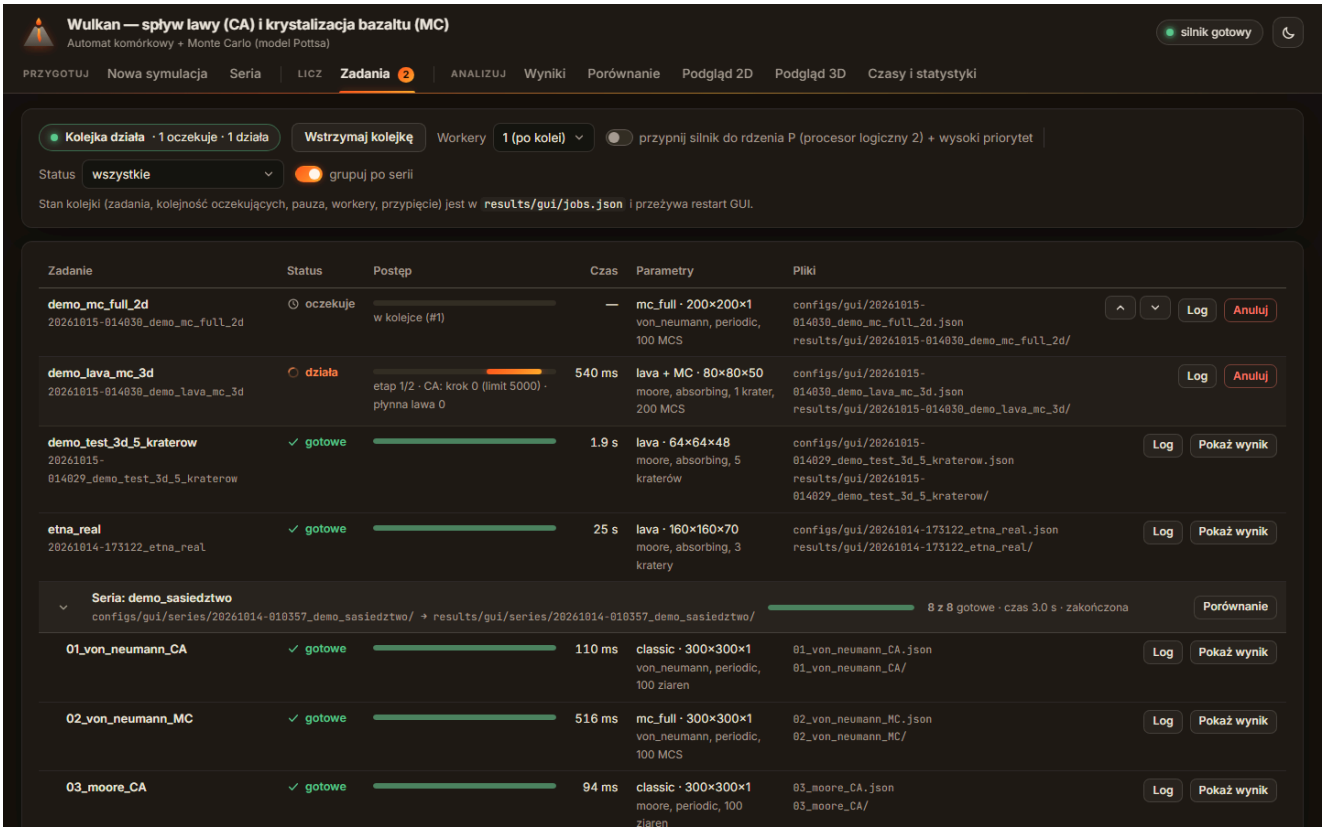
Błędny parametr (`space.nx = -5`): komunikat silnika przy polu:



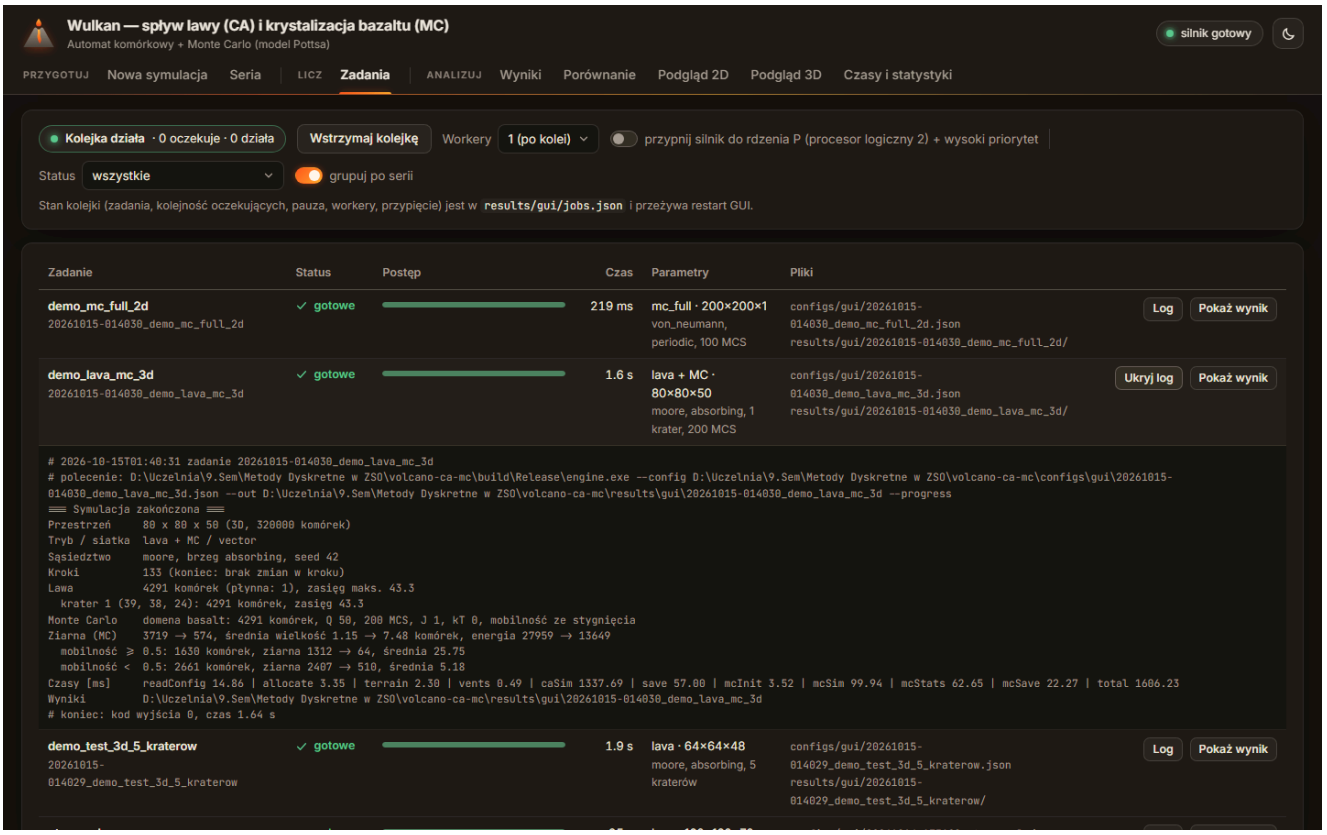
Po „Uruchom": ścieżki utworzonego configu i folderu wyników:



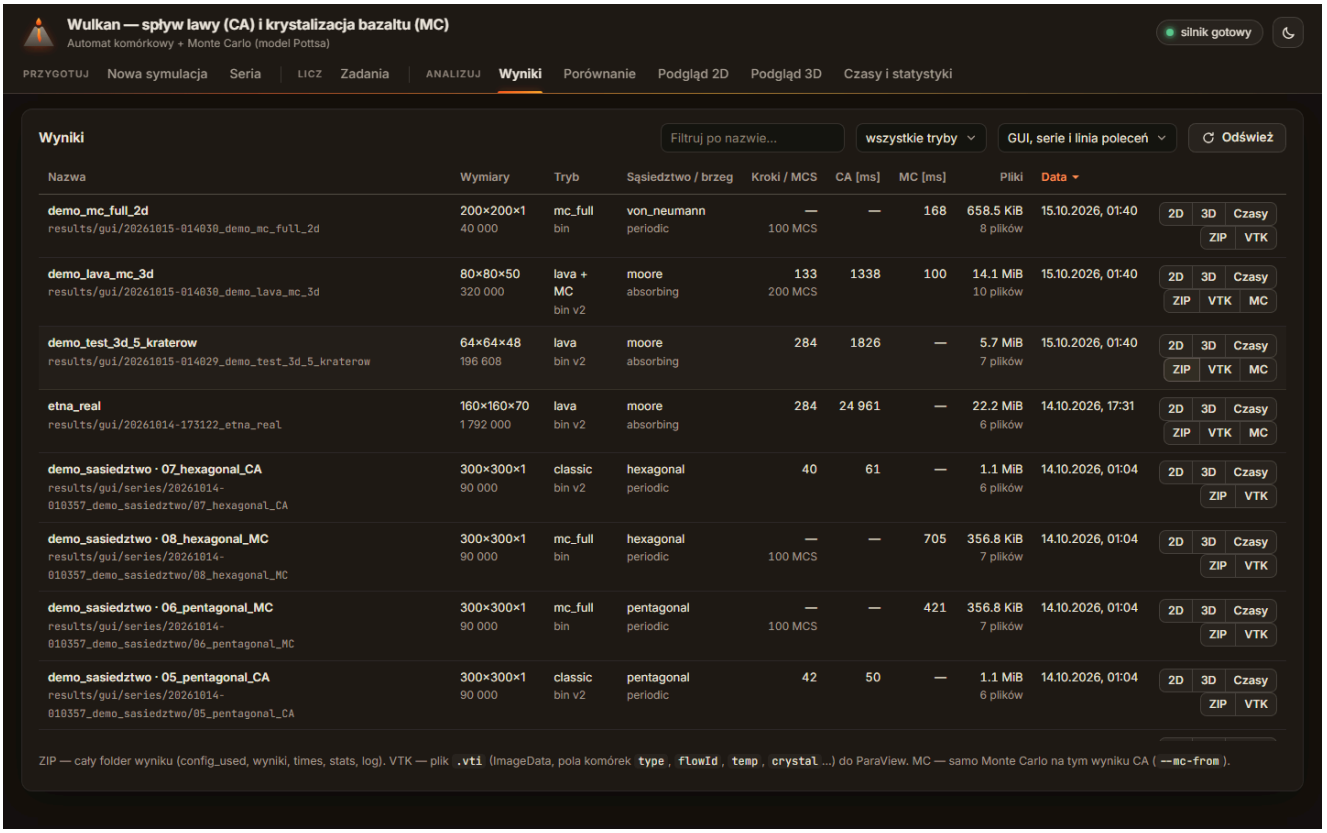
Zadania: trzy zadania dodane naraz: pierwsze działa (CA: krok i liczba komórek płynnej lawy; dla lawy koniec nie jest znany z góry, stąd animowany pasek), dwa czekają w kolejce FIFO:



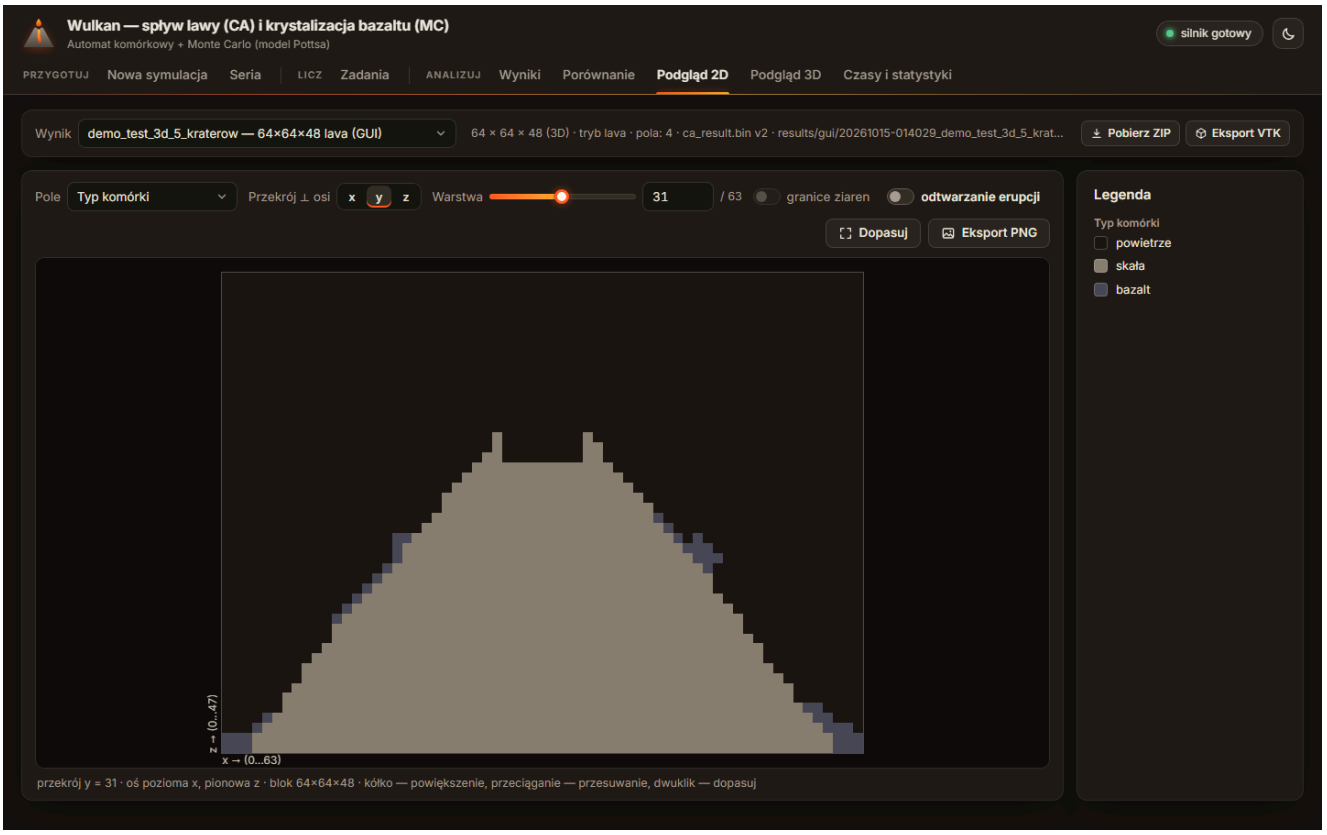
Po zakończeniu: statusy, czasy, parametry, pliki i rozwinięty log silnika:

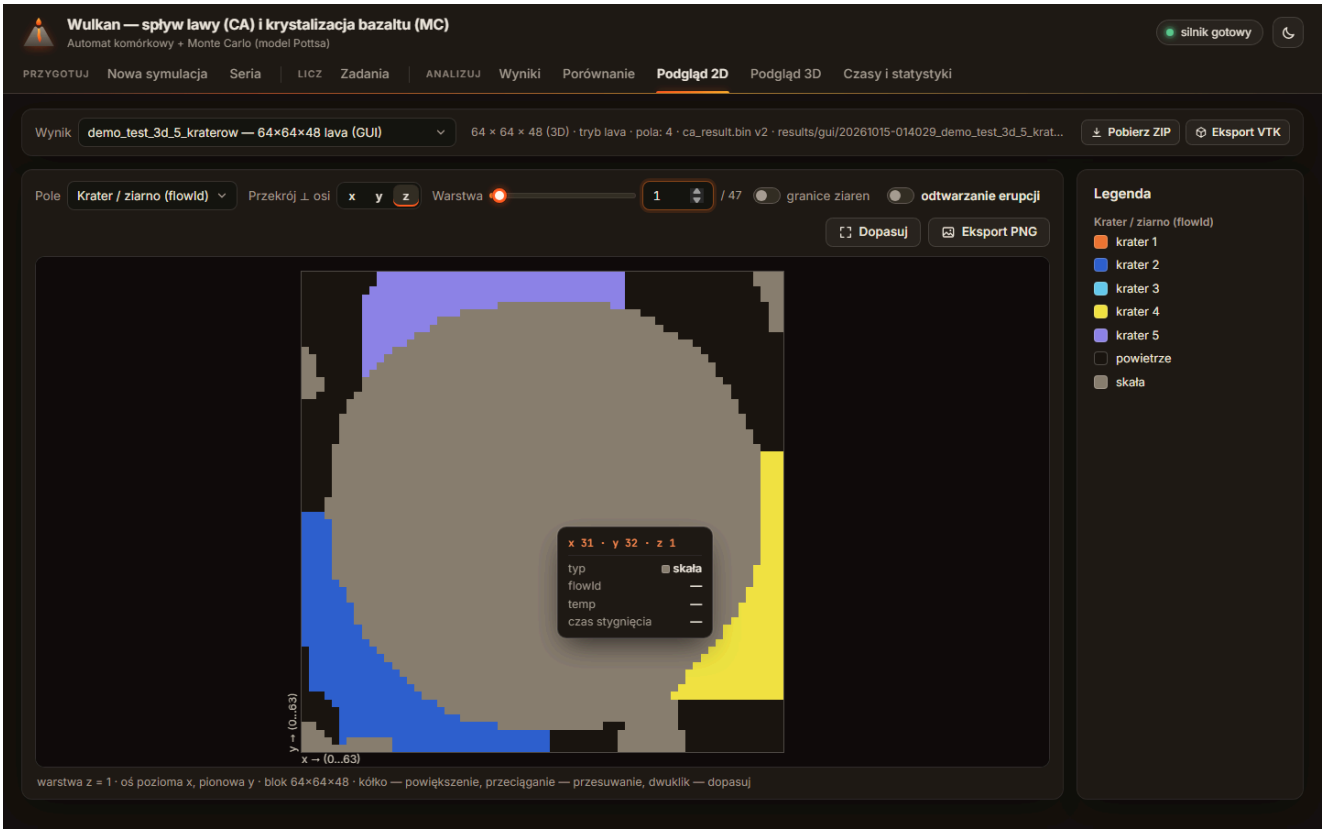


Wyniki: wyniki z GUI (`results/gui/…`) i z linii poleceń (`results/…`), z filtrem i sortowaniem:

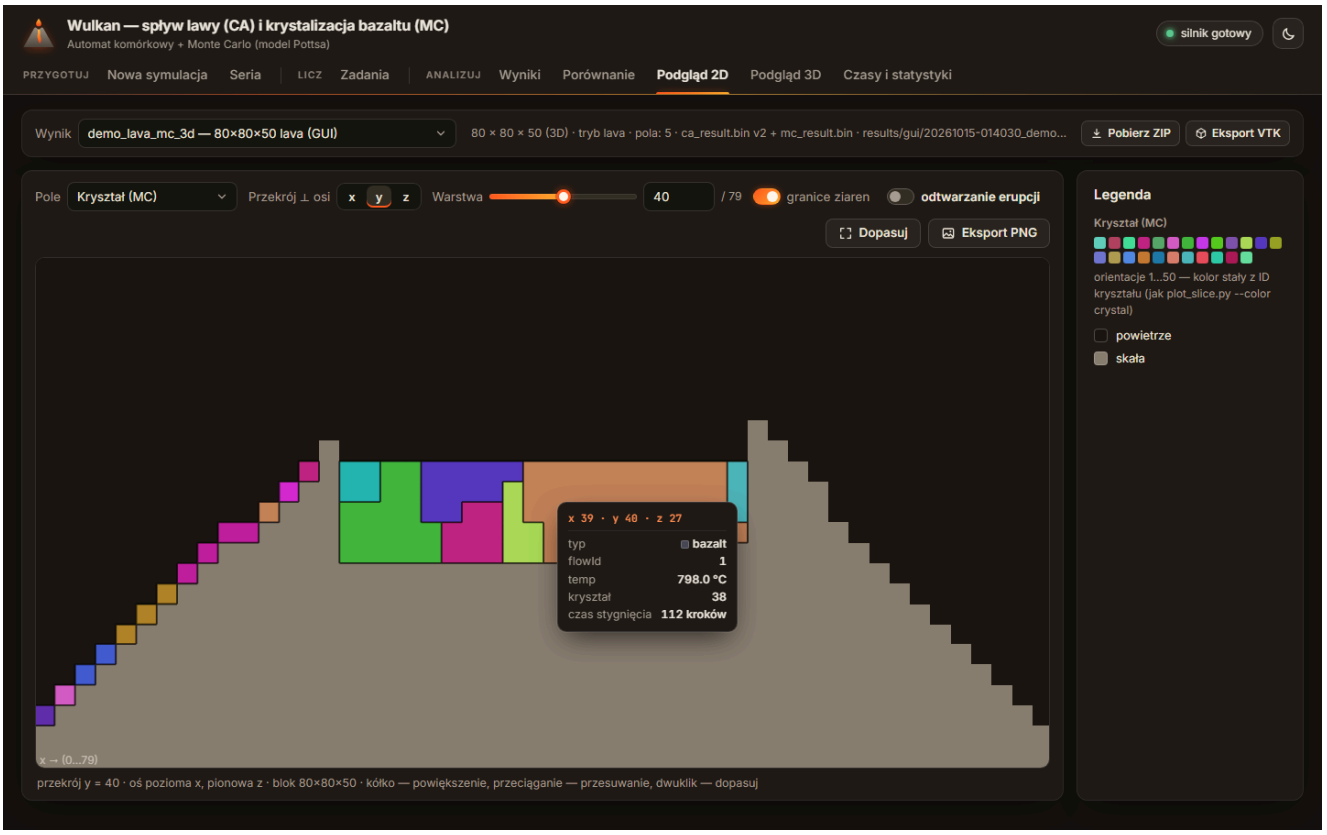


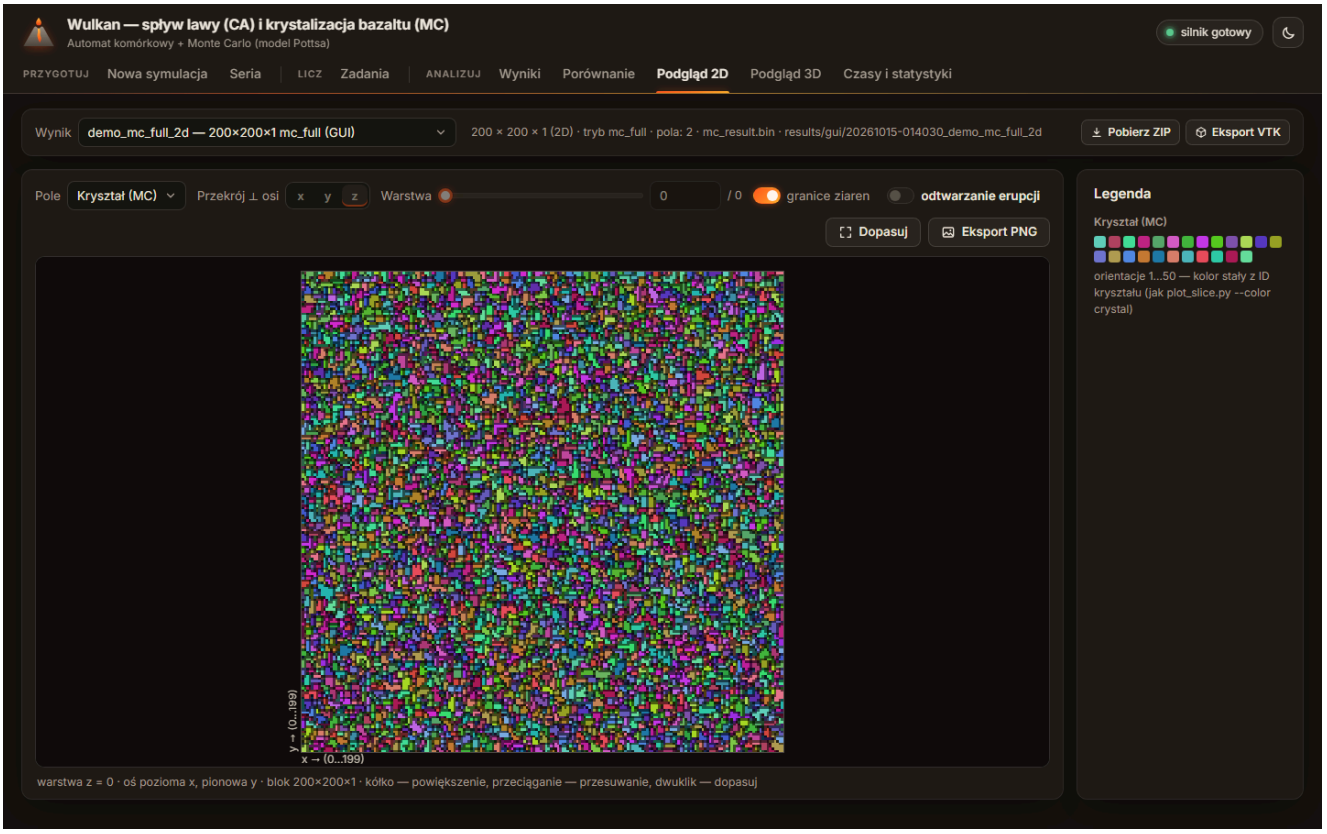
Podgląd 2D: przekrój y = 31 przez wulkan z 5 kraterami (typ komórki) i warstwa z = 1 kolorowana kraterem (`flowId`), z podpowiedzią:



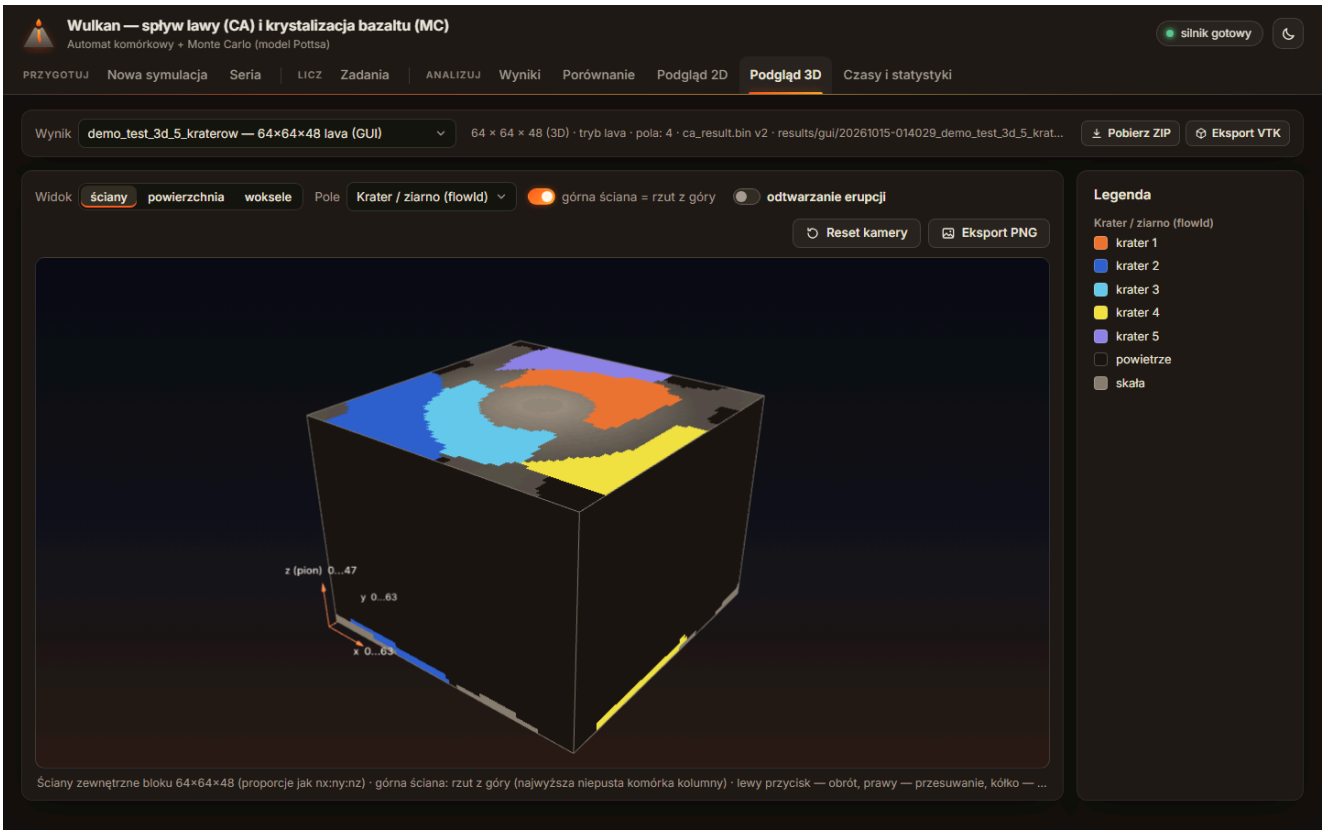


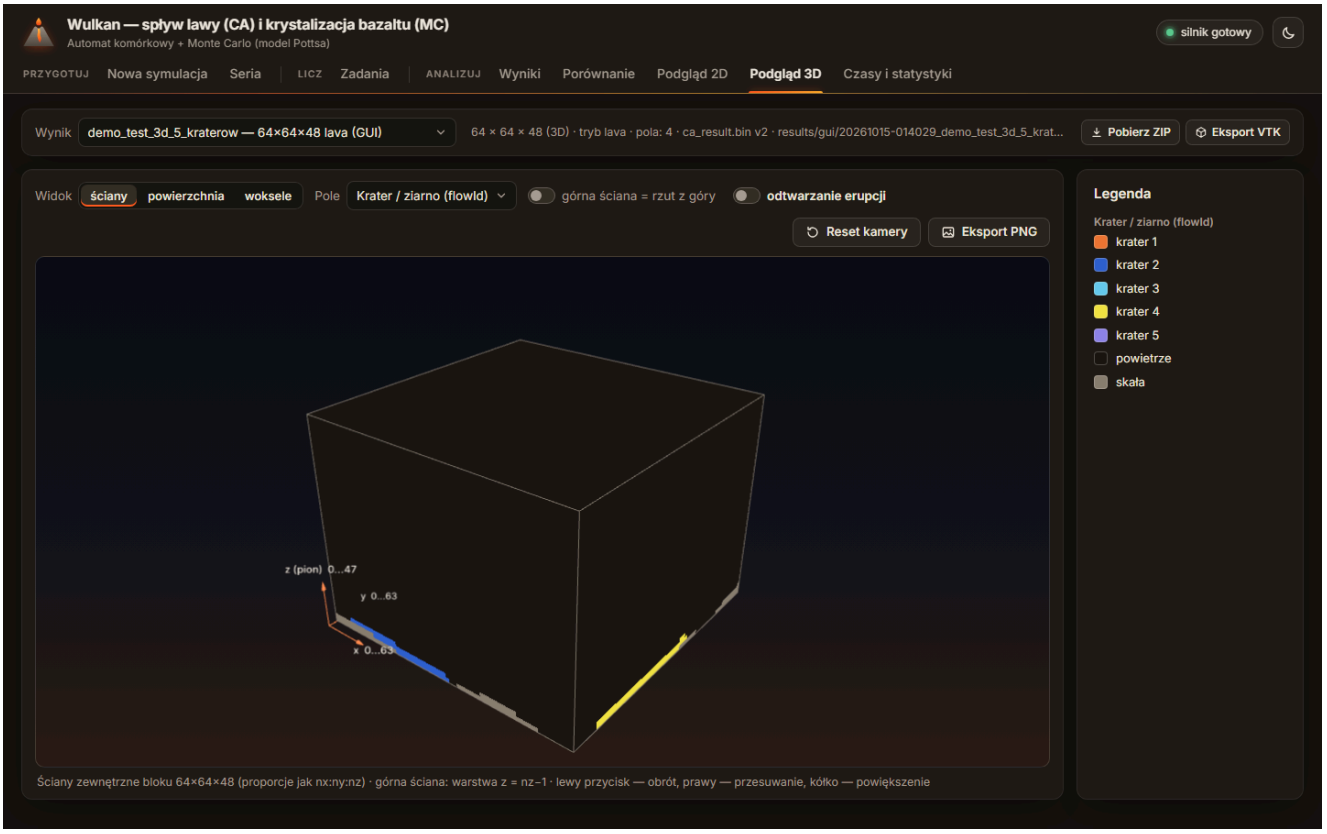
Kryształy MC: przekrój y = 40 przez krater `lava_mc_3d` (jezioro lawy z dużymi kryształami, drobne ziarna na zboczu, granice ziaren, podpowiedź z czasem stygnięcia) oraz `mc_full_2d` (MC na całej przestrzeni 200×200):



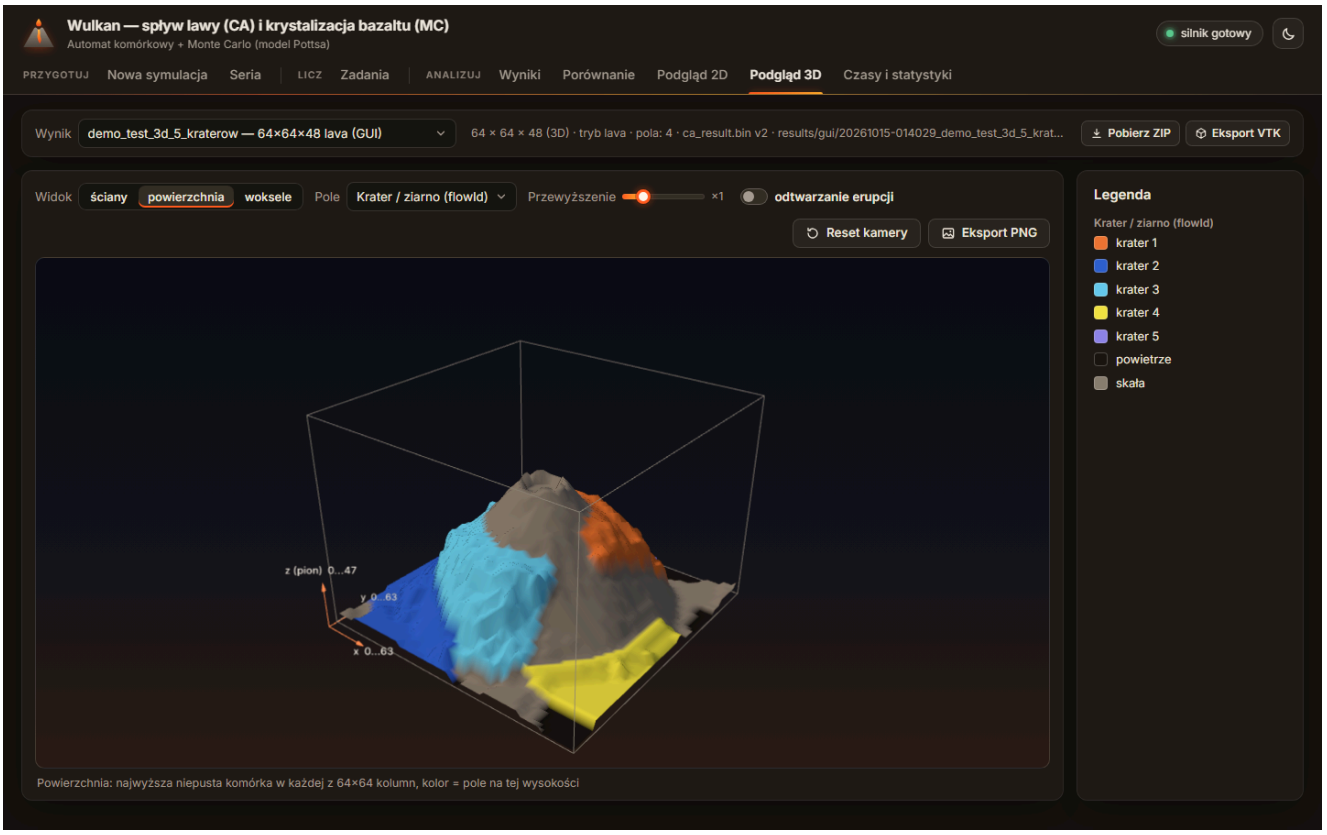


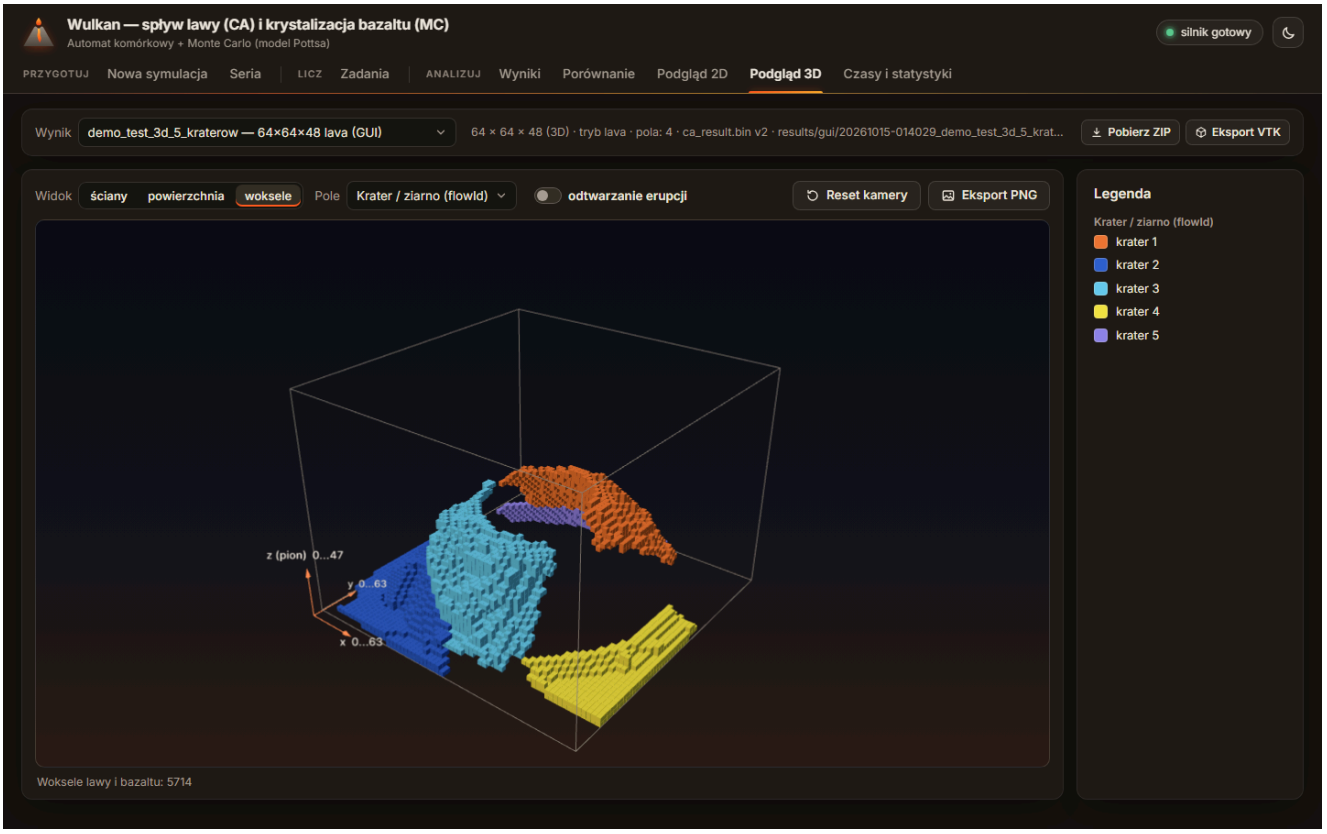
Podgląd 3D: ściany zewnętrzne bloku 64×64×48 (`flowId`). Górna ściana to domyślnie dla lawy rzut z góry (najwyższa niepusta komórka). Dosłowna warstwa z = nz−1 (drugi zrzut) to u wulkanu samo powietrze:



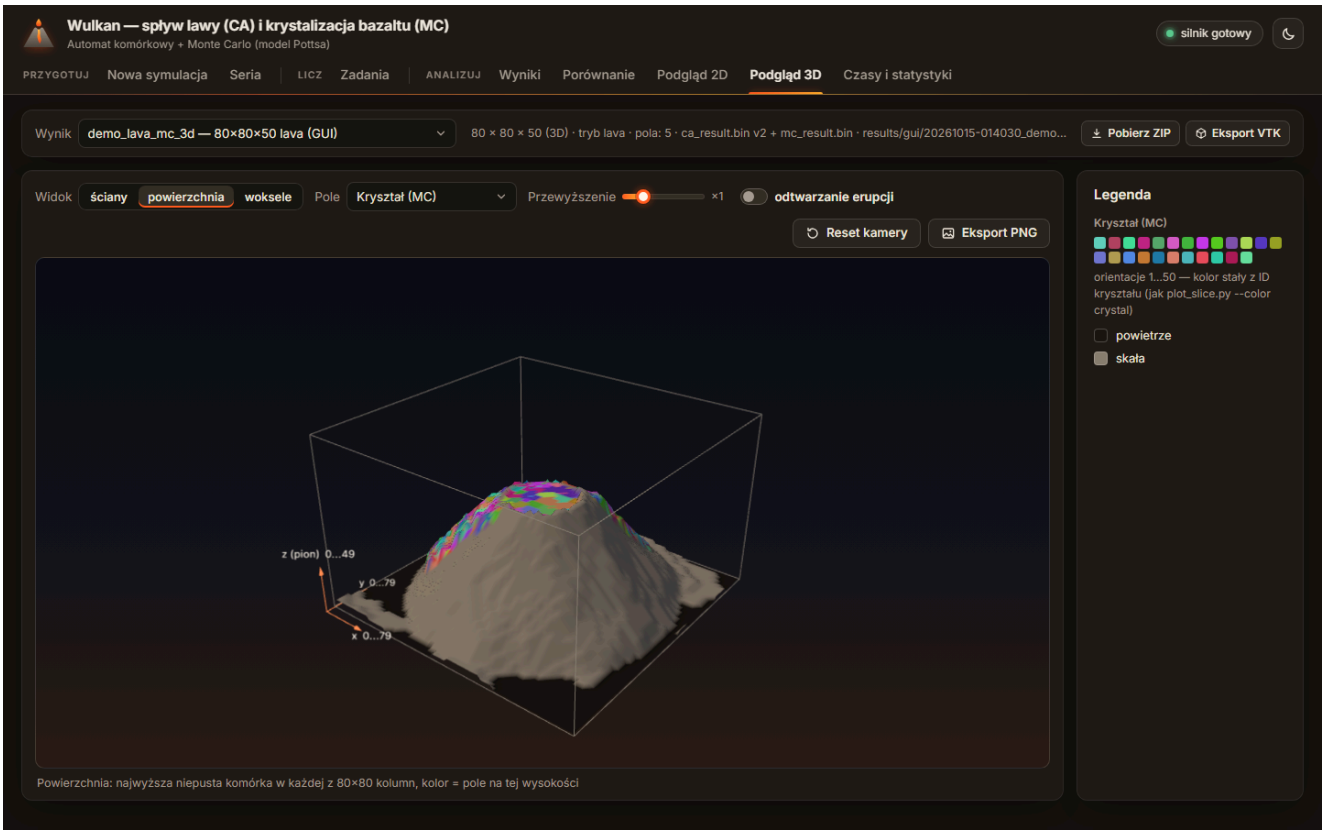


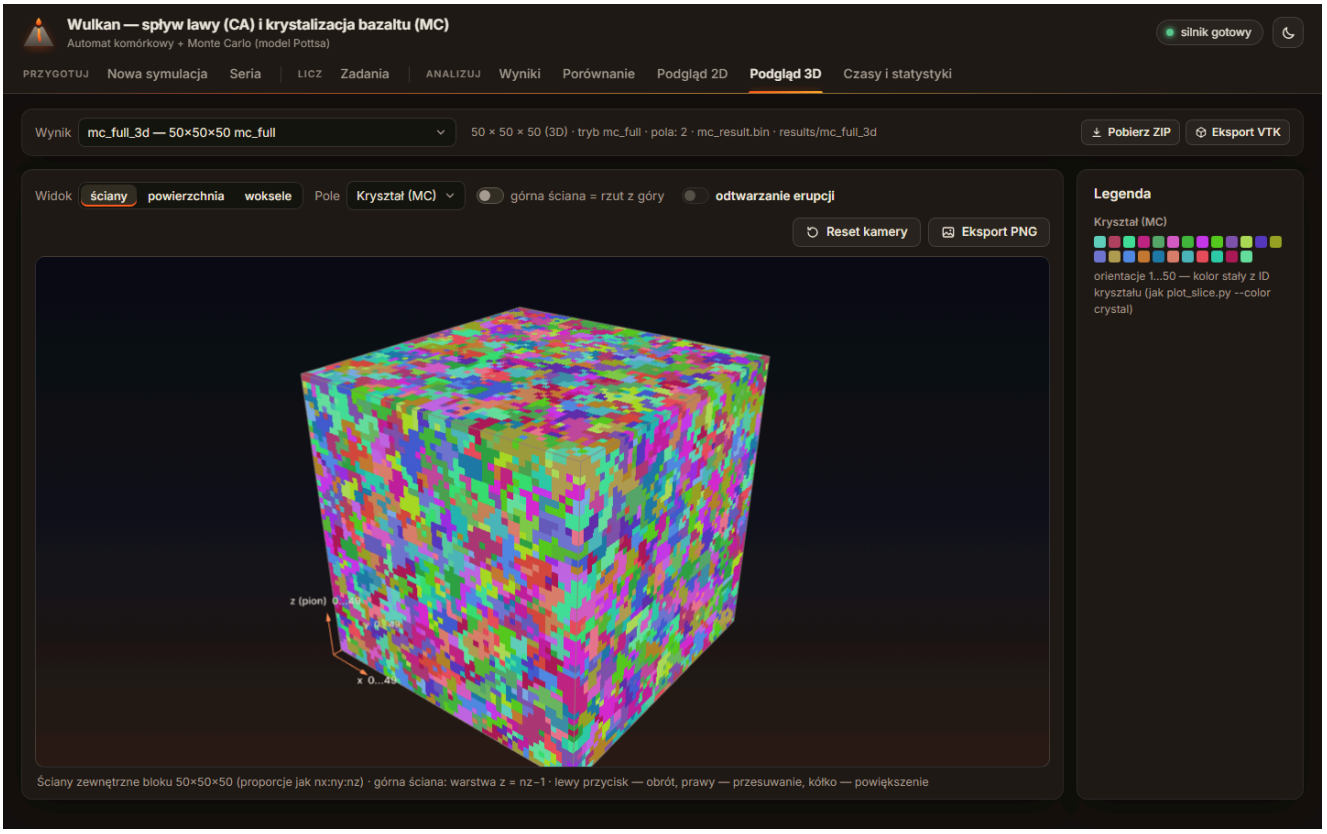
Powierzchnia (zbocze wulkanu z jęzorami lawy z 5 kraterów) i woksele lawy/bazaltu:



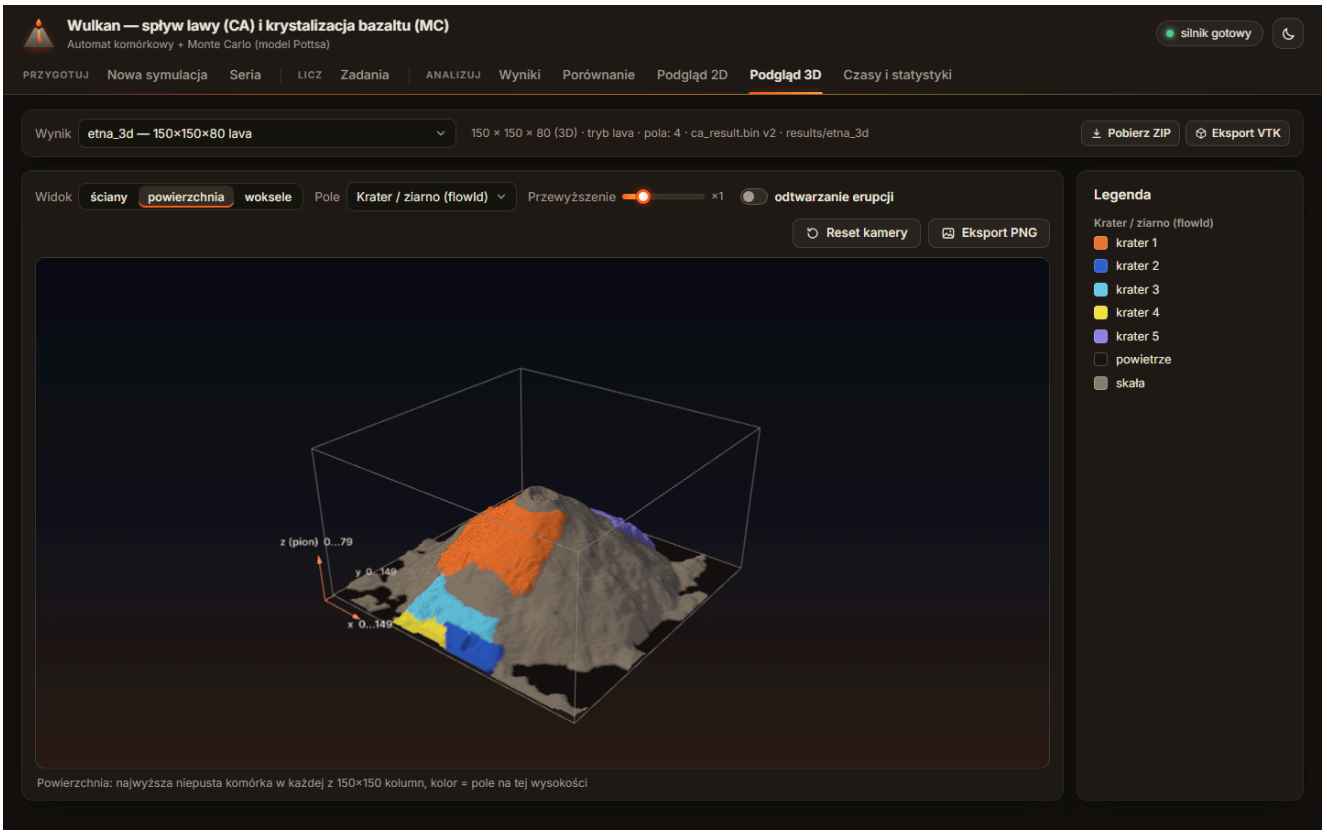


Kryształy MC na powierzchni (`lava_mc_3d`) i ściany kostki MC `full` 50^3 (kryształy):





Duży wynik: `etna_3d`, 150×150×80 = 1,8 mln komórek, powierzchnia:



Czasy i statystyki (`lava_mc_3d` z GUI): sekcje `times.json` z udziałem w całości, pamięć (pomiar i model), pliki, statystyki CA i MC (z efektem mobilności), wykresy liczby ziaren i średniej wielkości ziarna vs MCS oraz liczby komórek płynnej lawy w kolejnych krokach CA:



Pełna lista: `docs/img/gui_formularz.png`, `gui_formularz_blad.png`, `gui_formularz_uruchomiono.png`, `gui_zadania_postep.png`, `gui_zadania.png`, `gui_wyniki.png`, `gui_2d_typ.png`, `gui_2d_flowid.png`,

`gui_2d_krysztaly_lava.png`, `gui_2d_krysztaly_mc_full.png`, `gui_3d_sciany.png`, `gui_3d_sciany_warstwa.png`, `gui_3d_powierzchnia.png`, `gui_3d_woksele.png`, `gui_3d_krysztaly_powierzchnia.png`, `gui_3d_mc_full_sciany.png`, `gui_3d_etna_powierzchnia.png`, `gui_czasy.png`.

4. Jak uruchomić GUI

Wymagania: zbudowany silnik (`cmake --preset msvc-release`, `cmake --build --preset msvc-release` → `build\Release\engine.exe`), **Python ≥ 3.11** i przeglądarka (Chrome, Edge, Firefox; potrzebne są moduły ES i *import maps*). Internet nie jest potrzebny.

```
cd volcano-ca-mc
python -m pip install -r gui\requirements.txt      # pierwsze uruchomienie: fastapi, uvicorn, numpy, pillow
python -m gui                                      # serwer http://127.0.0.1:8000 + otwarcie przeglądarki
```

albo dwuklik **`gui\run_gui.bat`**, który sam doinstaluje pakiety, jeśli ich brakuje. Opcje: `python -m gui --port 8001 --no-browser`. Zatrzymanie: Ctrl+C w oknie konsoli. Alternatywnie: `python -m uvicorn gui.app:app` (port 8000).

Uwaga: jeśli Python jest zainstalowany dla wszystkich użytkowników (`C:\Python312`), zwykłe `pip install` nie ma prawa zapisu do `C:\Python312\Scripts`; trzeba użyć `python -m pip install --user -r gui\requirements.txt` (tak robi `run_gui.bat`).

Silnik jest wykrywany automatycznie: `build/Release/engine.exe`, `build-msvc-release/…`, `build-gcc-release/bin/engine`, a na końcu buildy Debug (z ostrzeżeniem, że są wolniejsze). Inną ścieżkę można podać w `gui/settings.json` (wzór: `gui/settings.example.json`; plik lokalny, poza repozytorium) albo w zmiennej `VOLCANO_ENGINE`. Port: `port` lub `VOLCANO_GUI_PORT`, liczba workerów: `workers` lub `VOLCANO_GUI_WORKERS`.

Testy i zrzuty ekranu (dodatkowe pakiety: `pip install -r gui\requirements-dev.txt`, `python -m playwright install chromium`):

```
python -m pytest gui\tests                         # 59 testów backendu (~2 s, wymaga zbudowanego silnika)
python gui\tests\screenshots.py                    # scenariusz w Chromium + zrzuty docs\img\gui_*.png
```

5. Zmiany w silniku (CLI)

Opcja	Działanie	Kod
`engine --print-defaults`	wypisuje `Config::defaultsJson()` (pełny domyślny config)	`app/ConfigCheck.*`, `main.cpp`
`engine --validate [--config f] [--set k=w]… [--mc-from d]`	tylko walidacja (bez symulacji i zapisu): JSON `{"ok": …, "errors": [...], "warnings": [...]}` na stdout, kod wyjścia 0 albo 1. Ten sam kod co uruchomienie (`Config::load`), a dodatkowo sprawdza to, co przy uruchomieniu wyszłoby dopiero w trakcie: plik mapy wysokości (`terrain.type = file`) oraz folder `--mc-from` i `mc.domain`	`app/ConfigCheck.*`
`engine … --progress`	co ~0,5 s (i zawsze na początku i końcu fazy) linia z `flush`: `PROGRESS ca <krok>/<maxSteps> active=<komórki LAVA>`, `PROGRESS mc <MCS>/<mcs>`, `PROGRESS save 0/1`. Bez flagi przebieg idzie dotychczasową ścieżką (`LavaCA::run()`)	`app/Progress.hpp`, `Simulation.cpp`, `McStage.cpp`

Decyzje: - `--validate` zawsze wypisuje JSON, także gdy config jest poprawny: `{"ok": true, "errors": [], "warnings": [...]}`. Pierwotny opis dopuszczał „pusty wynik” przy kodzie 0, ale wtedy ostrzeżenia nie miałyby gdzie trafić, a GUI i tak parsuje JSON. Walidacja konfiguracji jest „do pierwszego błędu” (tak działa `Config::fromJson`), więc `errors` ma zwykle jeden element. Po poprawieniu GUI pokaże następny. - `--progress` sprawdza zegar raz na krok CA / MCS: zegar czyta się poza pętlą po komórkach, a linia

trafia na wyjście najwyżej co 0,5 s. Dla CA podaje `ca.maxSteps` jako mianownik i liczbę aktywnych komórek, bo lawa kończy się zwykle dużo wcześniej (brak lawy / brak zmian). GUI pokazuje wtedy krok i płynną lawę na animowanym pasku. Dla `classic` pasek to odsetek wypełnionych komórek (`active / N`), dla MC dokładnie `MCS / mcs`. W MC raport jest poza pomiarem `mcSim` i `mcStats`. - Niedozwolone połączenia dają czytelny błąd argumentów: `--print-defaults` z innymi opcjami, `--validate` z `--benchmark` / `--progress` / `--timing-detail`, `--progress` z `--benchmark`. - Testy doctest: `engine/tests/test_gui_options.cpp`: 5 przypadków (parser opcji, `--print-defaults` = domyślny config i przechodzi walidację, walidacja: zła wartość, nieznany klucz, ostrzeżenie, brak pliku configu i mapy wysokości, `--mc-from` bez wyniku / brak folderu / zła domena, kod wyjścia i JSON; format i odstęp linii `ProgressReporter`; symulacja CA + MC z `--progress` daje bitowo te same `ca_result.bin`, `mc_result.bin`, `mc_stats.csv`, a ostatnie linie to stan końcowy) + 4 testy dymne ctest (`cli_print_defaults`, `cli_validate_ok`, `cli_validate_bad`, `cli_smoke_progress`).

Narzut `--progress` (`bench/ab_progress.py`: stary program sprzed GUI, nowy bez flagi, nowy z flagą; przemiennie, 11 powtórzeń, przypięte do rdzenia P, MSVC Release, `output.format = none`; czas symulacji z `times.json`):

Przypadek	stary [ms]	nowy [ms]	nowy + `--progress` [ms]	z / bez `--progress` (mediana par)
`test_3d` (CA lawa, `caSim`)	2 242 [2 018–2 314]	2 184 [1 988–2 380]	2 213 [2 003–2 252]	×1,007 [0,935–1,014]
`classic_ref` (CA classic 100³)	631 [583–688]	626 [580–685]	627 [571–660]	×0,999 [0,950–1,031]
`mc_full_3d` (MC 50³ Moore, `mcSim`)	991 [915–1 110]	921 [862–976]	933 [883–1 011]	×1,008 [0,945–1,098]
`lava_mc_3d` (CA + MC, `caSim`)	1 738 [1 571–1 799]	1 649 [1 487–1 758]	1 670 [1 512–1 772]	×1,007 [0,950–1,052]

Narzut mieści się w ~1% (mediana stosunku par −0,1…+0,8%), wyraźnie poniżej rozrzutu pomiaru (±5–10%). W MC raport jest całkowicie poza `mcSim`, więc +0,8% w `mc_full_3d` to sam szum. To pokazuje rozdzielczość tego pomiaru. Nowy program bez flagi był w tej sesji 1–7% szybszy od starego (zakresy się pokrywają). Kod pętli się nie zmienił, więc traktuję to jako szum/układ kodu, a nie przyspieszenie. Laptop był w tej sesji ~20% wolniejszy niż w pomiarach profilu aplikacji (np. `test_3d` 2,2 s zamiast 1,77 s), dlatego porównuję tylko przebiegi z tej samej sesji.

6. Weryfikacja

Sprawdzenie	Wynik
Build MSVC Release i Debug (`/W4`)	0 ostrzeżeń; `ctest` 11/11 w obu (7 dotychczasowych + 4 nowe testy dymne)
Build GCC 11.4 Release i Debug (WSL, `-Wall -Wextra`)	0 ostrzeżeń; `ctest` 11/11 w obu
`pytest gui/tests`	**59 passed** (~2,3 s), także z `-W default` bez ostrzeżeń
Wyniki bitowo identyczne jak przed GUI	`ca_result.bin`, `ca_result.txt`, `mc_result.bin`, `mc_result.txt`, `mc_stats.csv`, `stats.json` dla `test_2d`, `test_3d`, `lava_mc_2d`, `lava_mc_3d`, `mc_full_2d`, `classic_ref`: program sprzed GUI vs nowy bez `--progress` i z `--progress`: wszystkie identyczne (`cmp`). Nowy GCC (z `--progress`) vs stary MSVC: `test_2d`, `lava_mc_2d`, `mc_full_2d` identyczne
Narzut `--progress`	≤ ~1% (tabela wyżej)
Konsola przeglądarki (Playwright, Chromium)	brak błędów i ostrzeżeń JS w całym scenariuszu. Jedyne komunikaty to ostrzeżenia sterownika GL „GPU stall due to ReadPixels"; wywołuje je sam zrzut ekranu (`preserveDrawingBuffer`), skrypt je pomija

Scenariusz end-to-end (`gui/tests/screenshots.py` w prawdziwym Chromium, te same kliknięcia co ręcznie): 1. Start GUI → „Nowa symulacja" → preset `test_3d` → nazwa `demo_test_3d_5_kraterow`, kratery 3 → 5 → **Sprawdź** („Config poprawny") → `space.nx = -5` → **Sprawdź** (błąd „Parametr „space.nx" musi być z zakresu 1..100000, podano: −5" przy polu) → `nx = 64` → **Uruchom** → komunikat z `configs/gui/…json` i `results/gui/…/`. 2. Tak samo dla presetów `lava_mc_3d` i `mc_full_2d`: trzy zadania

w kolejce. Zakładka Zadania: pierwsze „działa” z krokiem CA, pozostałe „oczekuje”, potem po kolei „gotowe”: test_3d (5 kraterów) 2,2 s, lava_mc_3d 1,9 s, mc_full_2d 0,28 s (czas całego procesu silnika). Log silnika rozwija się w tabeli. 3. Wyniki → zadanie test_3d → Podgląd 2D: typ (przekrój y), `flowId` (warstwa z, 5 kraterów w legendzie, podpowiedź); lava_mc_3d → kryształy z granicami ziaren i powiększeniem; mc_full_2d → kryształy. 4. Podgląd 3D: ściany (górna jako rzut z góry i jako warstwa), powierzchnia, woksele; lava_mc_3d: kryształy na powierzchni; mc_full_3d: ściany kostki MC. 5. Czasy i statystyki lava_mc_3d: tabela czasów, pamięć, statystyki MC z `byMobility`, wykresy Chart.js. 6. **Duży wynik** `etna_3d` (1,8 mln komórek): podgląd 2D gotowy w **0,7 s** (metadane + przekrój), powierzchnia 3D w **0,9 s** (czasy z zegara skryptu, łącznie z oczekiwaniem na wyrenderowanie); przeglądarka się nie zawiesza.

Poza scenariuszem: wszystkie 15 wyników w `results/` (12 z wcześniejszych prac, w tym `classic_ref` 100^3, MC `full` i wynik `--mc-from` z typami komórek z folderu wejściowego, oraz 3 z GUI) wczytuje się we wszystkich widokach (przekrój każdego pola w 3 osiach, ściany w obu trybach, powierzchnia, woksele, komórka); skrypt po `ResultStore`, 0,01–0,10 s na wynik. Wyniki w `results/` mają też pliki `.bin`, więc ścieżkę txt sprawdziłem na kopiach samych plików txt: `classic_ref` (`ca_result.txt`, 1 mln wierszy): pierwsze wczytanie 0,53 s, potem z pamięci podręcznej 1 ms, przekrój identyczny jak z `.bin`; `mc_full_3d` (`mc_result.txt`): 0,04 s.

Testy pytest (`gui/tests/test_results.py`, `test_jobs.py`): - odczyt `ca_result.bin` v1 i v2, `ca_result.txt`, `mc_result.bin`/`.txt`: porównanie z tablicami sztucznego wyniku zapisanego według specyfikacji formatu (niezależnie od czytnika); pole `cooling`; wynik MC `full` bez CA; wynik `--mc-from` (typy z folderu `_mcFrom`); plik obcięty / zła sygnatura; - przekrój x / y / z dla każdego pola i 3 indeksów = wycinek numpy; próbkowanie dużej płaszczyzny; 6 ścian = numpy; mapa powierzchni = przeszukanie brute force (także pusta kolumna); górna ściana w trybie rzutu = mapa powierzchni; woksele i komórka; ścieżka memmap (próg 0) daje te same wycinki; - to samo przez HTTP (bajty + `X-Meta` → numpy); - zadania z prawdziwym silnikiem: `/api/defaults` = `engine --print-defaults`; walidacja (komunikaty po polsku, nieznany klucz, ostrzeżenie); błędny config → 422 i **nie powstaje żaden plik**; zadanie od początku do końca: config w `configs/gui/`, folder w `results/gui/`, status `gotowe`, kod 0, postęp, log bez linii PROGRESS, wynik na liście, wczytuje się, statystyki zgodne; kolejka FIFO (drugie zadanie startuje po pierwszym) i trwałość `jobs.json` (restart: „działa” → „przerwane”); anulowanie zadania oczekującego i działającego; MC na gotowym wyniku CA z zadania GUI (`mcFrom` → `engine --mc-from`: bez ostrzeżeń, wymiary z pliku, typy komórek z folderu wejściowego; przed policzeniem CA: 404; `mcFrom` spoza `results/`: 400); - bezpieczeństwo ścieżek: nazwa zadania `..\..\..\evil`, `../../etc/passwd`, `C:\Windows…` → tylko `[A-Za-z0-9_-]`, config i folder zawsze w `configs/gui/` i `results/gui/`; identyfikator wyniku `..`, `..\x`, `gui/../../x`, `C:\Windows`, `/etc`, `a/b/c` → błąd 400/404 (wynik nie może wyjść poza `results/`); POST bez `Content-Type: application/json` → 415; obcy nagłówek `Host` → 400.

7. Ograniczenia i co dalej

Ograniczenia: - GUI jest **lokalne i jednoużytkownikowe**: brak logowania, nasłuch tylko na `127.0.0.1`. Ochronę przed stronami z internetu dają tylko sprawdzenie nagłówka `Host` (DNS rebinding) i wymóg JSON w POST (bez CORS). Dwie instancje GUI na jednym katalogu projektu nadpisywałyby sobie `jobs.json`; `python -m gui` wykrywa zajęty port i otwiera działające GUI zamiast startować drugie. - **Postęp lawy** nie ma znanego końca (CA kończy się „brakiem lawy / brakiem zmian”), dlatego pasek jest animowany, a GUI pokazuje krok i liczbę komórek płynnej lawy. Szacunek czasu w formularzu to heurystyka rzędu wielkości. - **Anulowanie** zabija proces; w folderze zostają pliki zapisane do tej chwili (zwykle tylko `config_used.json`). - Ściany zewnętrzne bloku z wulkanem pośrodku to głównie powietrze (lawa dochodzi do brzegu tylko miejscami); tak wygląda dosłowny rzut ścian. Dlatego górna ściana ma opcję rzutu z góry, a widok „powierzchnia” pokazuje zbocze. - Powierzchnia 3D interpoluje kolor między sąsiednimi kolumnami (gładka siatka), więc granica jezora lawy jest lekko rozmyta. Dokładne granice pokazują przekrój 2D i woksele. - Pliki `.txt` powyżej 30 mln komórek nie są

wczytywane (komunikat: użyj `output.format = bin`). Pliki `.bin` > 128 MB są mapowane w pamięć; póki są oglądane, Windows nie pozwoli ich nadpisać innym uruchomieniem silnika do tego samego folderu (zadania z GUI zawsze dostają nowy folder). - Migawki `snapshotsEvery` nie były w pierwszej wersji GUI animowane; odtwarzanie erupcji krok po kroku doszło później (dokument „Prawdziwy teren i odtwarzanie erupcji", punkt 6). - `readConfig` w zadaniach z GUI wynosi 10–18 ms zamiast ~0,3 ms. To koszt pierwszego otwarcia świeżo zapisanego pliku przez system: z linii poleceń nowy plik zapisany z Pythona: 1–18 ms, ten sam plik drugi raz: 0,3 ms. Nie wpływa na `caSim` / `mcSim`. - GUI sprawdziłem tylko w Chromium (Playwright). Firefoksa i Safari nie testowałem; kod używa standardowych modułów ES, *import maps* i WebGL2, obsługiwanych przez aktualne wersje wszystkich trzech.

Co dalej: **kolejka, serie i dane wejściowe** (dokument „Kolejka zadań, serie, dane wejściowe"; architektura gotowa): - **kolejka wielu zadań**: już działa (FIFO, `workers` w ustawieniach); dojdzie widok „seria" w zakładce Zadania; - **generator serii**: wybór parametrów i zakresów (np. rozmiar × sąsiedztwo) → N configów → `JobManager.submit()` dla każdego (`POST /api/series`); - **wykres CA vs MC**: dane są w `times.json` każdego wyniku (`/api/result/details`); Chart.js jest w `vendor/`. Strona „Wyniki serii" zbierze `caSim` / `mcSim` wszystkich zadań serii, plus eksport PNG/CSV; - MC na wyniku CA z wybranego zadania (`mcFrom`) już działa w backendzie i formularzu; - raport techniczny (`docs/raport.md`) ze zrzutami GUI.

8. Pliki

- Silnik: `engine/src/app/ConfigCheck.hpp/.cpp`, `engine/src/app/Progress.hpp`, zmiany w `Cli.*`, `main.cpp`, `Simulation.*`, `McStage.*`; testy `engine/tests/test_gui_options.cpp`, testy dymne w `engine/CMakeLists.txt`.
- Backend `gui/`: `__main__.py`, `app.py`, `settings.py`, `engine.py`, `jobs.py`, `results.py`, `render.py`, `requirements.txt`, `requirements-dev.txt`, `run_gui.bat`, `settings.example.json`, `README.md`.
- Frontend `gui/static/`: `index.html`, `css/style.css`, `js/{main,api,form,estimate,palette,viewer2d,viewer3d,stats}.js`, `vendor/` (three.js r160 + `OrbitControls`, Chart.js 4.4.1, licencje, `README.md`).
- Testy: `gui/tests/conftest.py`, `test_results.py`, `test_jobs.py`, `screenshots.py`.
- `bench/ab_progress.py` (test A/B narzutu `--progress`), `docs/img/gui_*.png` (18 zrzutów), `docs/interfejs-graficzny.md`.
- `.gitignore`: `configs/gui/` (configi zadań, generowane jak `results/`) i `gui/settings.json` (ustawienia lokalne).